Name : Ananda Adiputra
ID : 2006462
AI LAB GROUP 2

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

7. Why must *h* be expressed in the same units as the edge costs?

8. Lab2 computes *h⋆* by running Dijkstra on the reversed graph from the goal. Why reversed?

9. Weighted A⋆ with *w = 1.5* changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

Answer :

1. BFS is an algorithm that is developed with the purpose of measuring the shortest distance to the goal by the total of edges it has to traverse. BFS treats every edge cost as 1, therefore it ignores the distances stored in the graph despite it being given to it.

2. BFS uses a queue while DFS uses a stack. Both algorithms have a similar code structure, therefore to differentiate both can be done by only changing two lines of code.

BFS	**DFS**

`# Frontier initialization`
`frontier = deque([start])`
`...`
	`# Frontier initialization`
	`frontier = [start]`
	`...`

`# Frontier dequeue` `frontier.dequeue()`	`# Frontier pop` `frontier.pop()`

Frontier is a collection of nodes that have been discovered but not yet explored during the traversal.

In the DFS algorithm, because of the concept of LIFO (Last in first out) as applied for Stack data structure, the next node of traversal is determined by the latest pushed neighbor nodes into the frontier. Therefore, the last added node will be explored by the algorithm DFS first and consequently the order of every neighbor node affects the algorithm's traversal.

3. Uniform Cost Search finds the shortest path by exploring nodes in order of their cumulative path cost using a priority queue. Usually, UCS tests if a state is the goal when it is popped from the queue, which guarantees the path found is the best. However, if the program tested for the goal when a state was first generated, it could mistakenly stop too early after finding a longer, more expensive path to the goal, completely overlooking a shorter and cheaper path waiting in the queue.

4. It is better to compare using an expanded approach, because it measures how many nodes the algorithms processed. For some different devices, it may have different hardware configurations from one to another, making one device perform faster or slower compared to the others, making the comparison of BFS and UCS from different laptops not reliable.

5. Greedy Best First Search only have one property which is f(n) = h(n) where h(n) is the estimated cost to the goal, while A* has 2 properties, f(n) = g(n) + h(n), where g(n) is the accumulated cost from the starting node and h(n) is the estimated cost to the goal.

   Q1 GBFS 12,500.5 km 2 expanded KUL -> LHR -> KEF
   Q1 A* 11,461.3 km 69 expanded KUL -> HKT -> LED -> HEL -> KEF

   According to the result based on Lab2 above, it explains the failure of GBFS in terms of finding the shortest path. Even though GBFS found the path quickly only by traversing 2 nodes, it failed to detect the possibility of another shortest past by only relying on heuristic. In contrast with A* that also measures g(n). Therefore, GBFS will eventually return a higher-cost path as a result of ignoring g(n).

6. UCS:
   $f(x) = g(x)$
   $f(x) = 8$
   $f(y) = g(y)$
   $f(y) = 6$
   $f(x) > f(y)$
   Since the state of y produces a smaller cost, y is expanded first

   GBFS:
   $f(x) = h(x)$
   $f(x) = 2$
   $f(y) = h(y)$
   $f(y) = 6$
   $f(x) < f(y)$
   Since the state of x produces a smaller cost, x is expanded first

   A*:
   $f(x) = g(x) + h(x)$
   $f(x) = 8 + 2 = 10$
   $f(y) = g(y) + h(y)$
   $f(y) = 3 + 6 = 9$
   $f(x) > f(y)$
   Since the state of y produces a smaller cost, y is expanded first

7. In A* that has properties of g(n) and h(n), both must be the same units therefore the total cost can be easily calculated, measured, and compared with another algorithm. Otherwise, mismatches of units may cause mathematical inaccuracy, invalidating the optimized path for the algorithm.

8. Heuristic is a cost estimation from a certain node to the target goal. Lab2 computes h* Using Dijkstra's algorithm backward is because in order to have cost estimation from every node to the target goal, it is easier to reverse the Dijkstra by starting from the target node ensuring computing cost-to-goal for all nodes in a single run.

9. Weighted A* for SIN→EZE does not change because of :
   - Non existing of another / alternative routes
   - When the heuristic (estimated cost to the goal) gets too large, the algorithm will automatically underestimate the g(n) accumulated cost by heavily relying on heuristic (h(n)) and behave like Greedy Best First Search (GBFS). Consequently, resulting in the cost for the route from SIN to EZE unchanged.